

변하는 MEC 연결을 그래프와 DQN 으로 읽기

Offloading Strategy Based on Graph Neural Reinforcement Learning in
Mobile Edge Computing

Tao Wang, Xue Ouyang, Dingmi Sun, Yimin Chen, Hao Li (2024)

M-GNRL: 두 hop GraphSAGE feature와 edge-score DQN으로 BS offloading을 선택

논문의 문제와 방법 범위

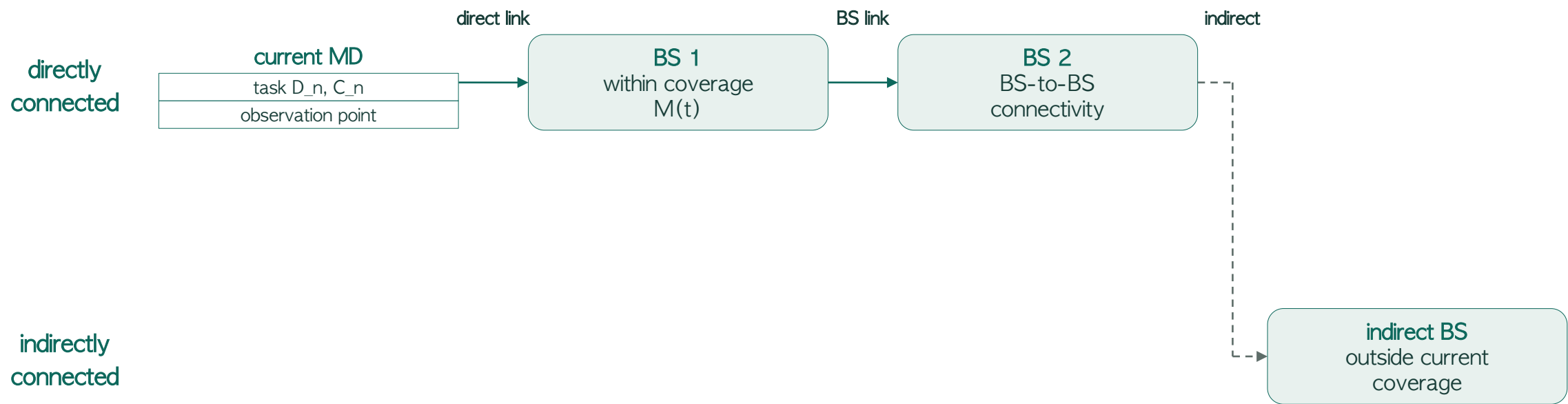
현재 MD의 이동으로 직접·간접 BS 관계가 달라질 때 system cost를 낮추는 offloading 전략

- MEC graph의 vertex는 MD와 BS이며 현재 MD가 observation point다
- task는 한 slot 시작에 하나씩 생기며 time·energy를 함께 cost로 둔다
- 두 hop GraphSAGE가 node feature를 만들고 DQN이 edge score로 BS를 고른다
- single-agent simulation이며 physical deployment나 multi-agent coordination은 다루지 않는다

관계 구조를 feature로 넣어 동적 MEC offloading을 학습하는 논문이다

MEC structural graph와 coverage

BS와 MD의 연결은 관측 MD의 위치가 바뀌면 달라지고 graph topology도 갱신된다



local과 offload의 cost 구성

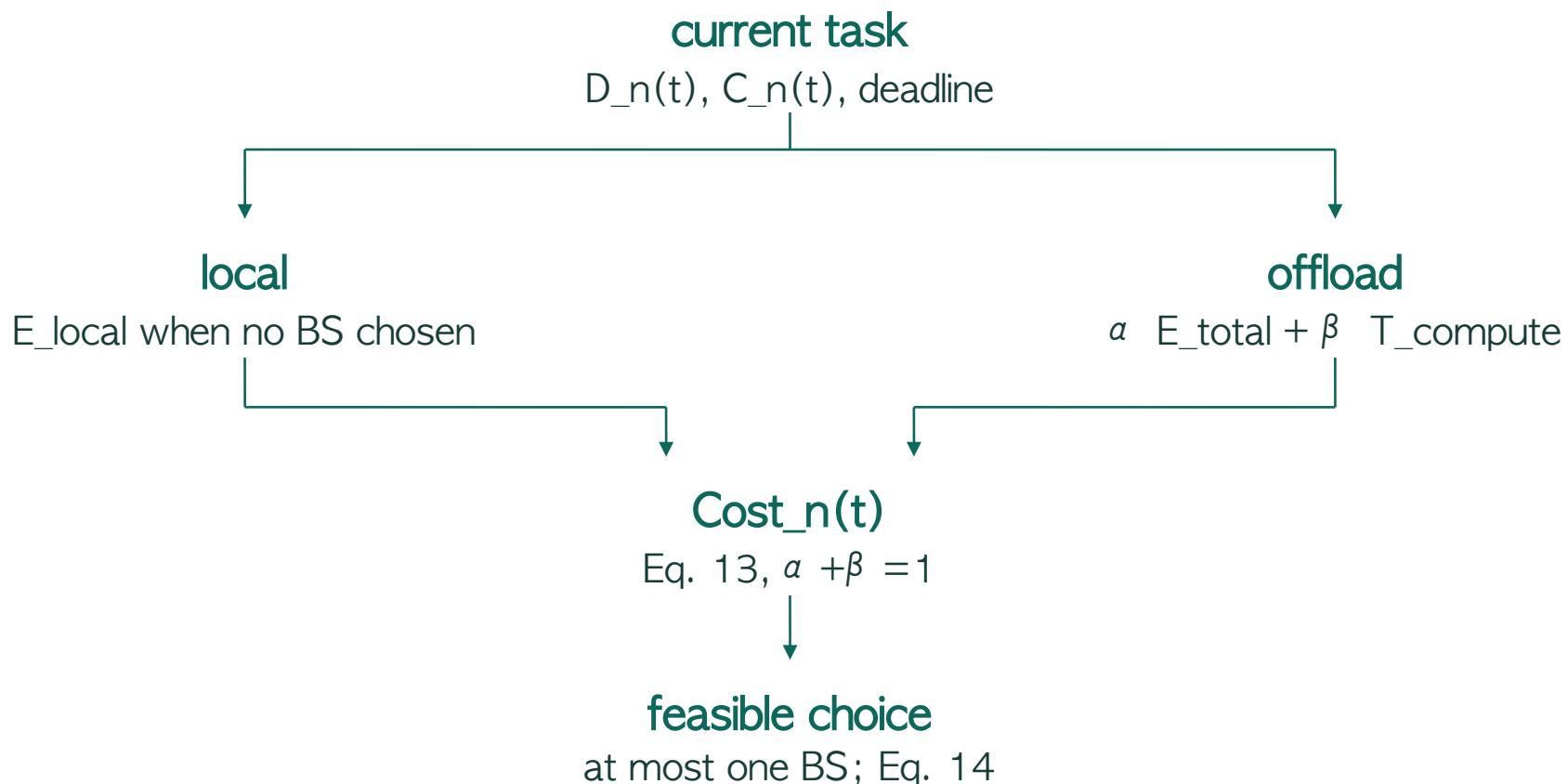
선택한 BS의 경로가 direct인지 indirect인지에 따라 transmission 항이 달라진다

항목	원문 정의	역할
rate	$R_{n,m}=B \log_2 (1+P_{n,m} g_n / (N_0+distance^{\theta}))$	direct upload rate
local	$T_{local}=C_n/f_{local}; E_{local}=P_n T_{local}$	device가 직접 실행
direct offload	$T_{trans}=D_n/R_{n,m}; E_{trans}=P_{n,m} T_{trans}$	MD에서 BS로 전송
indirect BS	forwarding time·energy를 추가	BS relay가 있는 경로
BS compute	$f_{com}=weight \times remaining\ resource$	BS load와 task 속성 반영

offload cost는 radio transmission과 BS execution을 함께 반영한다

cost와 multiple constraints

binary BS choice, BS frequency, deadline과 max execution time을 함께 제한한다



왜 graph feature가 필요한가

현재 MD는 가까운 node만 보면 간접 BS와 multi-hop 관계를 놓칠 수 있다

- 원문은 irregular MEC relation을 non-Euclidean graph로 모델링한다
- message passing은 이웃 node의 feature를 현재 node embedding에 반영한다
- 이 논문은 direct-indirect BS만 다루므로 two-hop information을 사용한다
- GNN output은 DQN의 environment와 edge-score 계산의 입력이 된다

GNN은 BS 후보의 관계 정보를 압축하는 feature stage다

두 hop GraphSAGE feature learning

sampled neighbors를 집계해 MD와 BS embedding을 업데이트한다

01	raw graph	$G_t = (V_g, \text{zeta})$
↓		
02	sample neighbors	MD-BS와 BS-BS
↓		
03	layer 1	neighbor aggregation
↓		
04	layer 2	two-hop context
↓		
05	embedding	updated h_n, h_m

layer 수 2는 이 논문의 direct-indirect BS 범위에 맞춘 선택이다

embedding 학습과 DQN 학습을 분리한다

GraphSAGE embedding loss와 DQN temporal-difference loss는 서로 다른 단계다

단계	입력·목표	업데이트
GraphSAGE	neighbor embedding과 negative samples	Eq. 20; Adam으로 W1, W2
edge feature	$\text{Con}(h_n, h_m) \rightarrow \text{MLP1} \rightarrow \text{MLP2} \rightarrow \text{softmax}$	Eq. 23
DQN	replay sample의 reward와 Q value	Eq. 26; gradient update Eq. 27

GNN message aggregation과 DQN policy learning을 같은 loss로 합치지 않는다

DQN state와 원문 reward

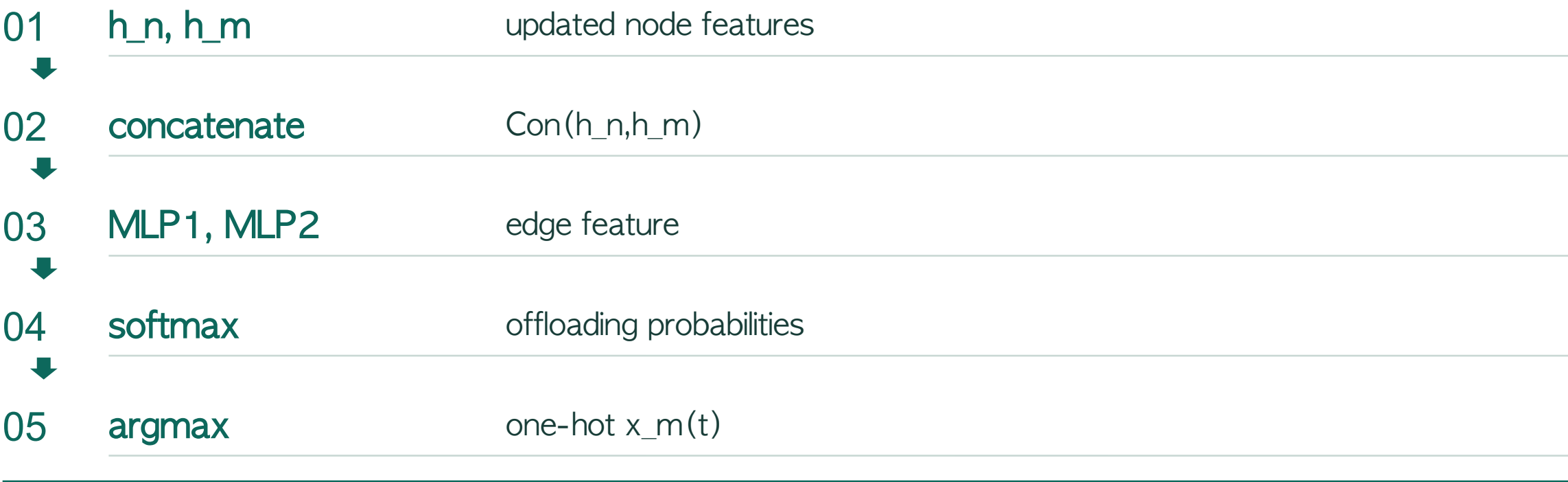
state는 task·deadline·BS load·remaining resource를 포함하며 reward 정의는 원문 그대로 읽는다

요소	원문 내용	주의
state s_t	$D_n, C_n, De_n, L_{\mu}(m), L_{\mu}(m)^s$	Eq. 22
on time	$r_t=1/Cost_n(t), 0 < T_e \leq \tau_n$	Eq. 21의 표기
otherwise	$r_t=.1(\tau_n-T_e)$	deadline/max time 관련
experience	(s,a,r,s') replay pool	Algorithm 1

reward의 부호나 cost-to-reward 변환을 텍에서 임의로 고치지 않는다

edge feature에서 one-hot BS action까지

updated MD와 BS feature의 softmax score 중 최대값을 택하는 방식이다



Eq. 23의 edge feature는 graph link 자체와 구별되는 decision score다

교육용 edge-score toy

아래 score는 원문 데이터가 아닌 Eq. 23-25를 재계산하는 가정이다

candidate BS	invented softmax score	one-hot x_m	선택
BS 1	.20	0	아님
BS 2	.65	1	max score
indirect BS 3	.15	0	아님
check	$.20+.65+.15=1$	one entry	BS 2

이 toy는 max-probability rule을 보일 뿐 paper experiment를 재현하지 않는다

Algorithm 1의 training loop

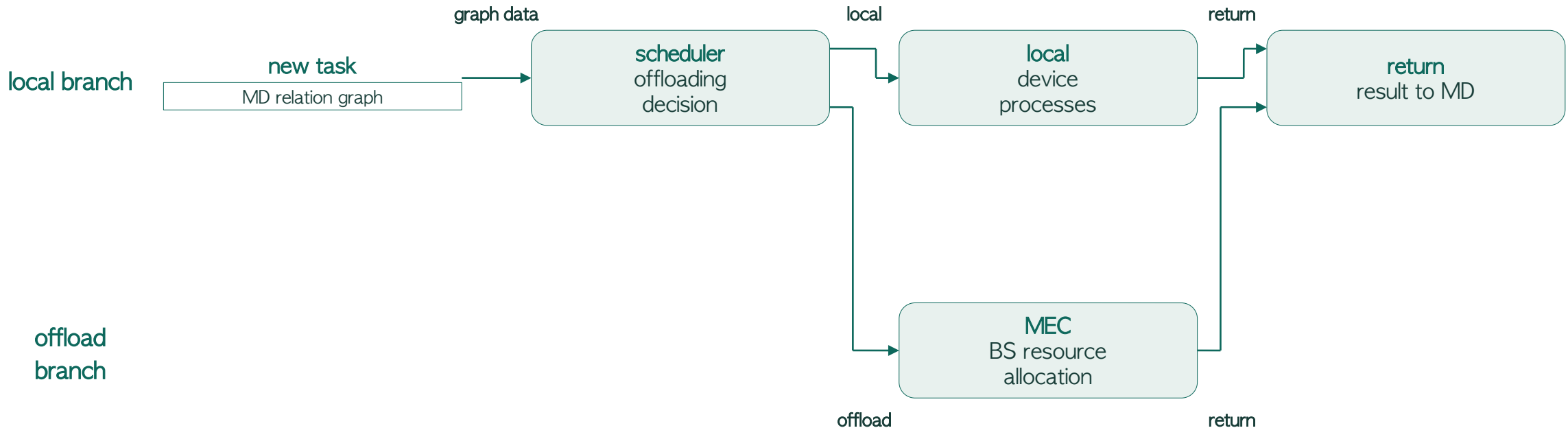
graph embedding을 만든 뒤 DQN replay loop에서 parameter ω 를 갱신한다

01	embedding	Eqs. 15-20; Adam
↓		
02	graph environment	learned graph input
↓		
03	state and action	Eqs. 22-24
↓		
04	store tuple	s, a, r, s' replay
↓		
05	mini-batch update	Eqs. 26-27

Algorithm 1은 pre-trained scheduler와 replay-based DQN training을 연결한다

학습된 scheduler 이후 task execution

Fig. 3은 graph data를 scheduler로 보내고, 결과를 local 또는 MEC processing path로 이어 준다



관측 MD 이동 뒤 adjacency-list update

Algorithm 2는 changed-set 입력 아래 vertex-table addition을 보이는 illustration이다



simulation graph records

실험 입력은 MD record, BS record, connection record의 세 종류다

record	원문 fields	graph 역할
MD	ID, D, C, deadline, priority, local CPU, slot	task node feature
BS	ID, F, P, load, remaining resource, range, slot	BS node feature
connection	BS ID, MD/BS ID, rate, distance, slot	edge relation and attributes

원문 실험은 graph input schema를 명시하지만 공개 dataset download는 제공하지 않는다

Table 2 simulation conditions

수치 결과는 bandwidth, task, CPU, replay setting이 정해진 simulation에서 나온다

범주	조건	값
network	bandwidth; slot sequence	4 MHz; T=80
task	size; CPU cycles	800–2000 kbytes; 1000–2500 Mcycles
compute	MD CPU; BS CPU	.5–1.5 GHz; 4–11 GHz
RL	episodes; batch; replay	1000/1200/1400; 100; 1500
RL	gamma; learning rate	.93/.95/.97/.99; .001

reported performance를 MEC deployment specification으로 일반화하면 안 된다

비교 baseline의 역할

각 baseline은 동일한 GNN-DQN variant가 아니라 서로 다른 policy family다

방법	원문 설명	비교 의미
LOCAL	device에서만 task 실행	offloading 없음
RANDOM	offload와 BS selection을 random	nonlearned reference
GNN-A2C	aerial EdgeloT graph RL framework	GNN plus A2C reference
Coop-UEC	drone cooperative DRL offloading	long-term reward DRL reference

M-GNRL의 결과는 이 네 방법 및 논문의 simulation 조건에 대한 비교다

discount factor 선택과 convergence

Fig. 7은 $\gamma=.97$ 을 이후 실험에 쓰게 한 empirical selection이고 Fig. 8은 조건부 convergence 비교다

방법 또는 setting	reported observation	조건
$\gamma=.97$	Fig. 7의 highest average reward	tested .93, .95, .97, .99
Coop-UEC	1050 episode 뒤 .35-.44	Fig. 8
GNN-A2C	920 iteration 뒤 .66-.78	Fig. 8
M-GNRL	950 iteration 뒤 .82-.97	Fig. 8

GNN-A2C는 약간 더 빠르지만 M-GNRL은 stable state에서 더 높은 reward를 보였다고 보고한다

arrival rate와 system cost 결과

task load, topology, coverage, BS frequency를 각각 바꾼 simulation 결과를 같은 주장으로 합치지 않는다

- Fig. 9a는 fixed BS resource에서 task volume이 커질 때 task arrival rate를 비교한다
- Fig. 9b는 direct·indirect BS 수와 MD 수가 다른 six topology를 비교한다
- Fig. 10은 topology, BS coverage, BS frequency에 따른 system cost를 제시한다
- 원문은 GNN-A2C가 Coop-UEC 대비 약 22.8%, M-GNRL이 GNN-A2C 대비 약 15.6% cost를 낮췄다고 쓴다

22.8%와 15.6%는 서로 다른 baseline에 대한 조건부 비교다

weight coefficient의 경계

time이나 energy 한쪽만 cost에 두면 Fig. 11에서 extreme cost와 uneven allocation이 관찰된다

setting	원문 비교	해석 경계
alpha=0, beta=1	time-side only	Fig. 11 conditions
alpha=1, beta=0	energy-side only	Fig. 11 conditions
alpha=.7, beta=.3	mixed setting	Fig. 11 condition
alpha=.3, beta=.7	mixed setting	Fig. 11 condition
alpha=.5, beta=.5	mixed setting	Fig. 11 condition

cost weights는 objective preference이며 paper가 보편적 최적 값을 제시하지 않는다

source-grounded limits

논문의 mechanism claim과 evidence boundary를 분리해 읽는다

- Graph update는 adjacency-list addition demonstration이며 deletion은 text에서 similar procedure라고만 말한다
- Algorithm 1은 single-agent task offloading이며 future work로 multi-agent collaboration·competition을 든다
- Eq. 13은 local option을 두지만 Eq. 23–25는 BS edge-score action만 구체화한다
- 결론은 simulation이 idealized setting에 제약된다고 밝히며 real-world test를 future work로 둔다

M-GNRL은 dynamic graph simulation 전략이며 deployed MEC guarantee가 아니다

M-GNRL을 한 흐름으로 읽기

moving MD relation을 graph로 갱신하고, two-hop feature와 DQN edge score를 거쳐 offloading choice를 만든다

01	MEC graph	MD-BS relation
↓		
02	GraphSAGE	two-hop embeddings
↓		
03	DQN	state, reward, replay
↓		
04	edge action	softmax and argmax
↓		
05	execution	local or MEC path

핵심 기여는 graph relation feature를 DQN offloading decision에 연결한 점이다